

COMP5318 Assignment 1: Rice Classification

Group number: 123

Student 1 SID: 550894694

Student 2 SID: ...

Student 3 SID: 550378183

1. Data Pre-processing

```
In [1]: # Import all libraries
import numpy as np
import pandas as pd
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import MinMaxScaler
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import AdaBoostClassifier, GradientBoostingClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, f1_score
```

```
In [2]: # Ignore future warnings
from warnings import simplefilter
simplefilter(action='ignore', category=FutureWarning)
```

```
In [3]: # Load the rice dataset: rice-final2.csv
from pathlib import Path

data_path = Path.cwd() / "rice-final2.csv"
rice_data = pd.read_csv(data_path)
rice_data.head()
```

```
Out[3]:
```

	Area	Perimeter	Major_Axis_Length	Minor_Axis_Length	Eccentricity
0	12573	461.4660034	192.9033508	84.57207489	0.898771763
1	12845	464.1210022	194.3322144	85.52433777	0.897951967
2	14055	488.7489929	207.7517548	87.25032806	0.907536149
3	14412	490.3240051	207.4761353	89.68951416	0.901735425
4	14658	477.1170044	189.5666351	99.99777985	0.849550545

```
In [4]: # Pre-process dataset
def preprocess_dataset(data):
    """Preprocess a rice-format dataset without hard-coding its sha
    features = data.iloc[:, :-1].replace("?", np.nan)
    labels = data.iloc[:, -1]

    features = features.apply(pd.to_numeric, errors="coerce")

    imputer = SimpleImputer(strategy="mean")
    imputed_features = imputer.fit_transform(features)

    scaler = MinMaxScaler()
    X_processed = scaler.fit_transform(imputed_features)

    y_processed = labels.map({"class1": 0, "class2": 1}).astype(int)

    return X_processed, y_processed

X, y = preprocess_dataset(rice_data)
```

```
In [5]: # Print first ten rows of pre-processed dataset to 4 decimal places
# A function is provided to assist

def print_data(X, y, n_rows=10):
    """Takes a numpy data array and target and prints the first ten

    Arguments:
        X: numpy array of shape (n_examples, n_features)
        y: numpy array of shape (n_examples)
        n_rows: numpy of rows to print
    """
    for example_num in range(n_rows):
        for feature in X[example_num]:
            print(f"{feature:.4f}", end=",")

        if example_num == len(X)-1:
            print(y[example_num], end="")
        else:
            print(y[example_num])

    print_data(X, y)
```

```
0.4628,0.5406,0.5113,0.4803,0.7380,0.4699,0.1196,1
0.4900,0.5547,0.5266,0.5018,0.7319,0.4926,0.8030,1
0.6109,0.6847,0.6707,0.5409,0.8032,0.6253,0.1185,0
0.6466,0.6930,0.6677,0.5961,0.7601,0.6467,0.2669,0
0.6712,0.6233,0.4755,0.8293,0.3721,0.6803,0.4211,1
0.2634,0.2932,0.2414,0.4127,0.5521,0.2752,0.2825,1
0.8175,0.9501,0.9515,0.5925,0.9245,0.8162,0.0000,0
0.3174,0.3588,0.3601,0.3908,0.6921,0.3261,0.8510,1
0.3130,0.3050,0.2150,0.5189,0.3974,0.3159,0.4570,1
0.5120,0.5237,0.4409,0.6235,0.5460,0.5111,0.3155,1
```

2. Build Classifiers

- Part 1: Logistic Regression, Naïve Bayes
- Part 2: KNN, Decision Tree, Ada Boost, Gradient Boost, Random Forest, SVM

Part 1: Cross-validation without parameter tuning

```
In [6]: ## Setting the 10 fold stratified cross-validation
cvKFold=StratifiedKFold(n_splits=10, shuffle=True, random_state=0)

# The stratified folds from cvKFold should be provided to the class
```

```
In [7]: # Logistic Regression
logr_classifier = LogisticRegression(
    solver="liblinear",
    random_state=0
)

logr_scores = cross_val_score(
    logr_classifier,
    X,
    y,
    cv=cvKFold,
    scoring="accuracy"
)

logr_cv_accuracy = logr_scores.mean()
```

```
In [8]: # Naïve Bayes
nb_classifier = GaussianNB()
nb_scores = cross_val_score(nb_classifier, X, y, cv=cvKFold, scoring="accuracy")
nb_cv_accuracy = nb_scores.mean()
```

Part 1 Results

```
In [9]: # Print results for each classifier in part 1 to 4 decimal places
print(f"LogR average cross-validation accuracy: {logr_cv_accuracy:.4f}")
print(f"NB average cross-validation accuracy: {nb_cv_accuracy:.4f}")
```

LogR average cross-validation accuracy: 0.9386
 NB average cross-validation accuracy: 0.9264

Part 2: Cross-validation with parameter tuning

```
In [10]: # KNN
# parameters you may consider
k = [1, 3, 5, 7]
p = [1, 2]

knn_classifier = KNeighborsClassifier()
knn_param_grid = {
    "n_neighbors": k,
```

```
    "p": p
}
```

```
In [11]: # Decision Tree
# parameters you may consider
max_depth = [3, 5, 7, 10]
min_samples_split = [2, 5, 10]
min_samples_leaf = [1, 2, 4]

decision_tree_classifier = DecisionTreeClassifier(random_state=0)
decision_tree_param_grid = {
    "max_depth": max_depth,
    "min_samples_split": min_samples_split,
    "min_samples_leaf": min_samples_leaf
}
```

```
In [12]: # Ada Boost
# parameters you may consider
n_estimators = [50, 100, 150]
learning_rate = [0.1, 0.2, 0.3, 0.5]

ada_boost_classifier = AdaBoostClassifier(random_state=0)
ada_boost_param_grid = {
    "n_estimators": n_estimators,
    "learning_rate": learning_rate
}
```

```
In [13]: # Gradient Boost
# parameters you may consider
max_depth = [1, 3, 5, 7]
n_estimators = [50, 100, 150]
learning_rate = [0.1, 0.2, 0.3, 0.5]

gradient_boost_classifier = GradientBoostingClassifier(random_state=0)
gradient_boost_param_grid = {
    "max_depth": max_depth,
    "n_estimators": n_estimators,
    "learning_rate": learning_rate
}
```

```
In [14]: # Random Forest
# You should use RandomForestClassifier from sklearn.ensemble with
# parameters you may consider
n_estimators = [10, 30, 60, 100]
max_leaf_nodes = [6, 12]

random_forest_classifier = RandomForestClassifier(
    criterion="entropy",
    max_features="sqrt",
    random_state=0
)
random_forest_param_grid = {
    "n_estimators": n_estimators,
    "max_leaf_nodes": max_leaf_nodes
}
```

```
In [15]: # SVM
# parameters you may consider
C = [0.01, 0.1, 1, 5]
gamma = [0.01, 0.1, 1, 10]
# optional
kernel = ["rbf"]

svm_classifier = SVC(random_state=0)
svm_param_grid = {
    "C": C,
    "gamma": gamma,
    "kernel": kernel
}
```

Part 2: Results

```
In [16]: # Perform Grid Search with 10-fold stratified cross-validation (GridSearchCV)
# The stratified folds from cvKFold should be provided to GridSearchCV

# This should include using train_test_split from sklearn.model_selection
# Print results for each classifier here. All the reported results

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=0
)

def run_grid_search(classifier, parameter_grid):
    """Fit a classifier using the required stratified grid search."""
    grid_search = GridSearchCV(
        estimator=classifier,
        param_grid=parameter_grid,
        cv=cvKFold,
        scoring="accuracy",
        n_jobs=-1
    )
    grid_search.fit(X_train, y_train)
    return grid_search

knn_grid_search = run_grid_search(knn_classifier, knn_param_grid)
decision_tree_grid_search = run_grid_search(
    decision_tree_classifier, decision_tree_param_grid
)
ada_boost_grid_search = run_grid_search(ada_boost_classifier, ada_boost_param_grid)
gradient_boost_grid_search = run_grid_search(
    gradient_boost_classifier, gradient_boost_param_grid
)
random_forest_grid_search = run_grid_search(
    random_forest_classifier, random_forest_param_grid
)
svm_grid_search = run_grid_search(svm_classifier, svm_param_grid)
```

```

def print_grid_search_results(name, grid_search):
    """Print the required best parameters and accuracy measures."""
    best_model = grid_search.best_estimator_
    test_accuracy = accuracy_score(y_test, best_model.predict(X_test))

    print(f"{name} best parameters: {grid_search.best_params_}")
    print(f"{name} cross-validation accuracy: {grid_search.best_score}")
    print(f"{name} test set accuracy: {test_accuracy:.4f}")
    print()

print_grid_search_results("KNN", knn_grid_search)
print_grid_search_results("Decision Tree", decision_tree_grid_search)
print_grid_search_results("Ada Boost", ada_boost_grid_search)
print_grid_search_results("Gradient Boost", gradient_boost_grid_search)
print_grid_search_results("Random Forest", random_forest_grid_search)
print_grid_search_results("SVM", svm_grid_search)

random_forest_best_model = random_forest_grid_search.best_estimator_
random_forest_predictions = random_forest_best_model.predict(X_test)
random_forest_macro_f1 = f1_score(
    y_test, random_forest_predictions, average="macro"
)
random_forest_weighted_f1 = f1_score(
    y_test, random_forest_predictions, average="weighted"
)

print(f"Random Forest macro average F1 score: {random_forest_macro_f1}")
print(f"Random Forest weighted average F1 score: {random_forest_weighted_f1}")

```

KNN best parameters: {'n_neighbors': 7, 'p': 2}
 KNN cross-validation accuracy: 0.9375
 KNN test set accuracy: 0.9250

Decision Tree best parameters: {'max_depth': 3, 'min_samples_leaf': 1, 'min_samples_split': 2}
 Decision Tree cross-validation accuracy: 0.9357
 Decision Tree test set accuracy: 0.9429

Ada Boost best parameters: {'learning_rate': 0.2, 'n_estimators': 150}
 Ada Boost cross-validation accuracy: 0.9455
 Ada Boost test set accuracy: 0.9429

Gradient Boost best parameters: {'learning_rate': 0.1, 'max_depth': 1, 'n_estimators': 50}
 Gradient Boost cross-validation accuracy: 0.9446
 Gradient Boost test set accuracy: 0.9429

Random Forest best parameters: {'max_leaf_nodes': 6, 'n_estimators': 30}
 Random Forest cross-validation accuracy: 0.9411
 Random Forest test set accuracy: 0.9429

SVM best parameters: {'C': 5, 'gamma': 1, 'kernel': 'rbf'}
 SVM cross-validation accuracy: 0.9429
 SVM test set accuracy: 0.9321

Random Forest macro average F1 score: 0.9414
 Random Forest weighted average F1 score: 0.9427

Test your code

```
In [17]: #load the test dataset to test out your model

RUN_TEST_BEFORE = False

if RUN_TEST_BEFORE:

    test_path = Path.cwd() / "test-before.csv"
    test_data = pd.read_csv(test_path)

    # ----- Preprocessing checks -----
    X_check, y_check = preprocess_dataset(test_data)

    # The code must work for datasets with different numbers of
    # examples and features.
    assert X_check.shape[0] == test_data.shape[0]
    assert X_check.shape[1] == test_data.shape[1] - 1
    assert y_check.shape[0] == test_data.shape[0]

    # No missing or infinite values should remain.
    assert np.isfinite(X_check).all()

    # Min-max scaling should produce values in [0, 1].
```

```

assert (X_check >= -1e-10).all()
assert (X_check <= 1 + 1e-10).all()

# Class labels should be converted to 0 and 1.
assert set(np.unique(y_check)).issubset({0, 1})

# ----- Part 1 checks -----
cv_check = StratifiedKFold(
    n_splits=10,
    shuffle=True,
    random_state=0
)

part1_models = [
    LogisticRegression(solver="liblinear", random_state=0),
    GaussianNB()
]

for model in part1_models:
    scores = cross_val_score(
        model,
        X_check,
        y_check,
        cv=cv_check,
        scoring="accuracy"
    )

    assert len(scores) == 10
    assert np.isfinite(scores).all()

# ----- Part 2 checks -----
X_check_train, X_check_test, y_check_train, y_check_test = train_test_split(
    X_check,
    y_check,
    test_size=0.2,
    stratify=y_check,
    random_state=0
)

test_models = [
    ("KNN", KNeighborsClassifier(), knn_param_grid),

    ("Decision Tree",
     DecisionTreeClassifier(random_state=0),
     decision_tree_param_grid),

    ("Ada Boost",
     AdaBoostClassifier(random_state=0),
     ada_boost_param_grid),

    ("Gradient Boost",
     GradientBoostingClassifier(random_state=0),
     gradient_boost_param_grid),

    ("Random Forest",
     RandomForestClassifier(

```

```

        criterion="entropy",
        max_features="sqrt",
        random_state=0
    ),
    random_forest_param_grid),

    ("SVM",
     SVC(random_state=0),
     svm_param_grid)
]

for name, classifier, parameter_grid in test_models:

    grid_search = GridSearchCV(
        estimator=classifier,
        param_grid=parameter_grid,
        cv=cv_check,
        scoring="accuracy",
        n_jobs=-1
    )

    grid_search.fit(X_check_train, y_check_train)

    predictions = grid_search.best_estimator_.predict(X_check_t

    # Verify that a prediction is produced for every test examp
    assert len(predictions) == len(y_check_test)

    # Predictions should only contain the two encoded classes.
    assert set(np.unique(predictions)).issubset({0, 1})

print("All preprocessing and classifier runnability checks pass

```

3. Reflection and Discussion

The classifiers performed quite similarly, but there were still some differences in their results. In Part 1, Logistic Regression achieved a higher average cross-validation accuracy (**0.9386**) than Naïve Bayes (**0.9264**). A possible reason is that Naïve Bayes assumes conditional independence between features, while some of the geometric measurements in the rice dataset may be related. Therefore, Logistic Regression appears to fit this dataset better than Naïve Bayes.

Among the tuned models in Part 2, AdaBoost achieved the highest cross-validation accuracy (**0.9455**), followed closely by Gradient Boosting (**0.9446**), SVM (**0.9429**), Random Forest (**0.9411**), KNN (**0.9375**) and Decision Tree (**0.9357**). On the test set, Decision Tree, AdaBoost, Gradient Boosting and Random Forest all achieved **0.9429**, while SVM achieved **0.9321** and KNN **0.9250**. We should not judge the models only from one test split, because cross-validation averages performance across multiple folds.

For example, the Decision Tree had a higher test accuracy than its cross-validation accuracy, which may be affected by the particular train-test split. AdaBoost and Gradient Boosting had very similar cross-validation and test results, suggesting relatively stable generalisation in this experiment.

The selected hyperparameters also show the trade-off between model complexity and generalisation. For KNN, **k = 7** and **p = 2** were selected. Using several neighbours makes the model less sensitive to individual noisy examples than a very small k. The Decision Tree selected **max_depth = 3**, which limits the size of the tree and helps reduce overfitting. Gradient Boosting selected **max_depth = 1**, **learning_rate = 0.1** and **50 estimators**, so it combines a number of simple weak learners rather than relying on one complex tree. The good results from AdaBoost, Gradient Boosting and Random Forest are also consistent with the idea that ensemble methods can produce more stable predictions by combining multiple learners. For SVM, the best parameters were an **RBF kernel**, **C = 5** and **γ = 1**, which control the flexibility of the nonlinear decision boundary.

GridSearchCV was useful because it selected these parameter combinations using stratified cross-validation instead of choosing them manually. However, we did not record results from the same Part 2 models using default parameters, so we cannot measure exactly how much the tuning improved their accuracy. For Random Forest, the macro F1 score (**0.9414**) and weighted F1 score (**0.9427**) were also very close to its test accuracy (**0.9429**), suggesting reasonably consistent performance across the two classes.

Overall, there was no single model that was clearly better in every result. Among the tuned models, **AdaBoost had the highest cross-validation accuracy**, while several models shared the highest test accuracy. Considering both results, **AdaBoost and Gradient Boosting showed the most consistently strong performance**, although the differences between the best-performing models were small.

AI Acknowledgement

Generative AI (ChatGPT) was used to assist with code drafting, notebook formatting, and wording for the written discussion. The final notebook was reviewed by the group and checked against the assignment requirements.